

3rd Semester cse Branch

OPERATING SYSTEM (OS)

PYQ QUESTION

EDUCATION
PLUS+



Last 10 Years | BEU B.Tech CSE 3rd Sem

Operating System - Important Questions (PYQs)

Bihar Engineering University | B.Tech CSE 3rd Semester



Unit 1

Introduction

Define Operating System. What are the main functions of an operating system?

An Operating System is system software that acts as an interface between the user and computer hardware, and manages system resources.

Main Functions

1. Process Management – Manages and schedules processes
2. Memory Management – Allocates and deallocates memory
3. File Management – Organizes and manages files
4. Device Management – Controls input/output devices
5. Security – Protects the system from unauthorized access
6. User Interface – Provides an interface for user interaction

What is a System Call? Give examples of different types of system calls.

- A system call is an interface through which a user program requests a service from the operating system kernel.
- Process Control: `fork()`, `exec()`, `exit()`, `wait()` — used to create, execute, and terminate processes.
- File Management: `open()`, `read()`, `write()`, `close()` — used to manipulate files on the system.
- Device Management: `ioctl()`, `read()`, `write()` — used to request and control peripheral devices.
- Communication: `pipe()`, `shmget()`, `mmap()` — used for inter-process communication and message passing.

What is the difference between user mode and kernel mode?

Feature	User Mode	Kernel Mode
Access Level	Limited access to resources	Full access to system resources
Purpose	Runs user applications	Runs core OS functions
Direct Hardware Access	Not allowed	Allowed
System Calls	Required to access OS services	Not required
Security	More secure (restricted)	Less restricted (high privilege)

What are the advantages and disadvantages of a Monolithic kernel?

A monolithic kernel is an OS design where all core services (process, memory, file system, drivers) run in a single kernel space.

Advantages:

1. High Performance – Fast execution due to direct communication
2. Efficient Resource Use – Fewer context switches
3. Simple Design – Easier structure compared to microkernel

Disadvantages:

1. Less Secure – A bug can crash the whole system
2. Hard to Maintain – Large and complex codebase
3. Low Modularity – Difficult to update or modify components separately

Explain the different structures of an Operating System (Monolithic, Layered, Microkernel). Compare their advantages and disadvantages.

1. Monolithic Structure

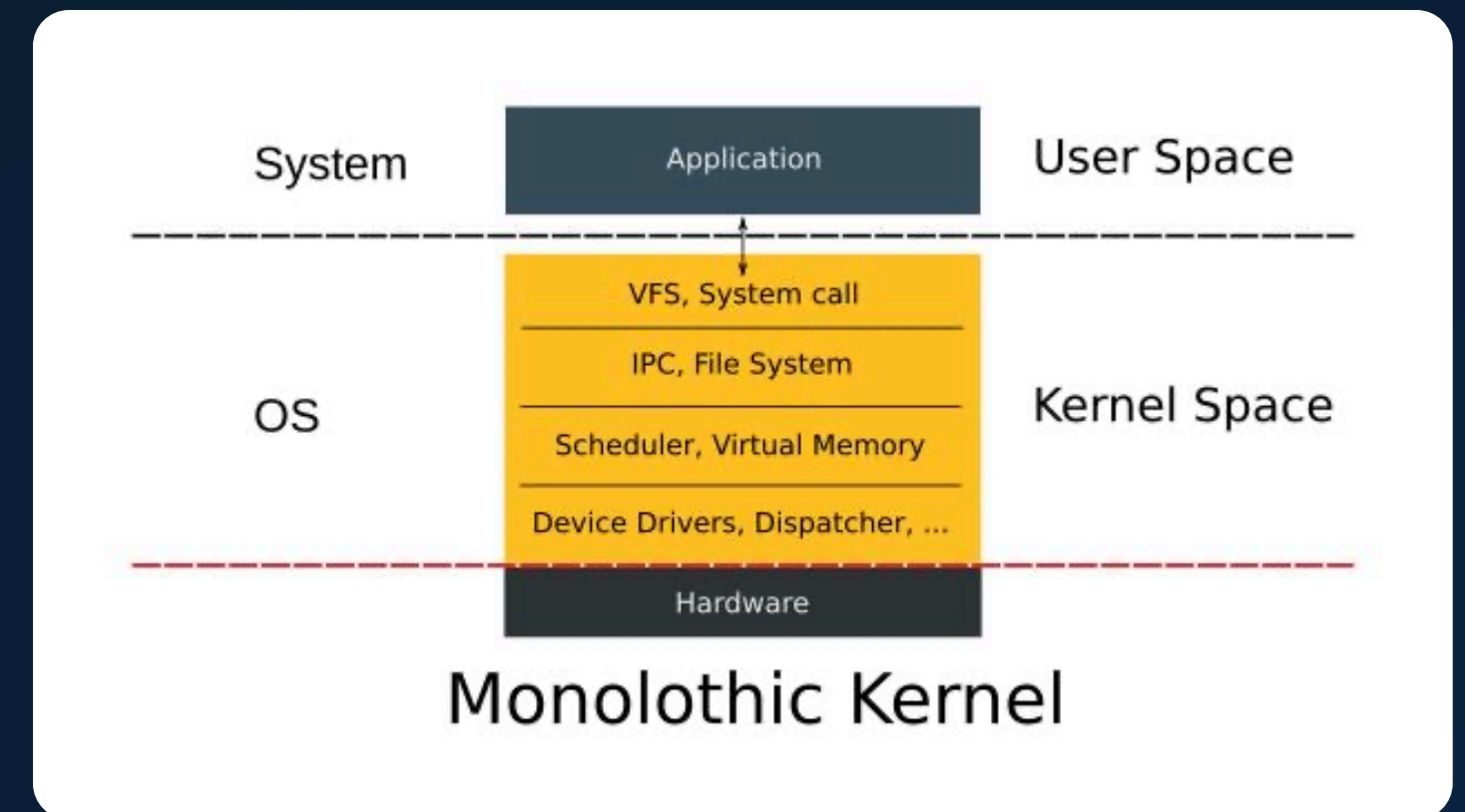
- All OS services run in a single kernel space
- Components directly communicate with each other

Advantages:

- High performance (fast execution)
- Efficient resource usage

Disadvantages:

- Poor security (one bug can crash entire system)
- Hard to maintain and debug



Explain the different structures of an Operating System (Monolithic, Layered, Microkernel). Compare their advantages and disadvantages.

2. Layered Structure

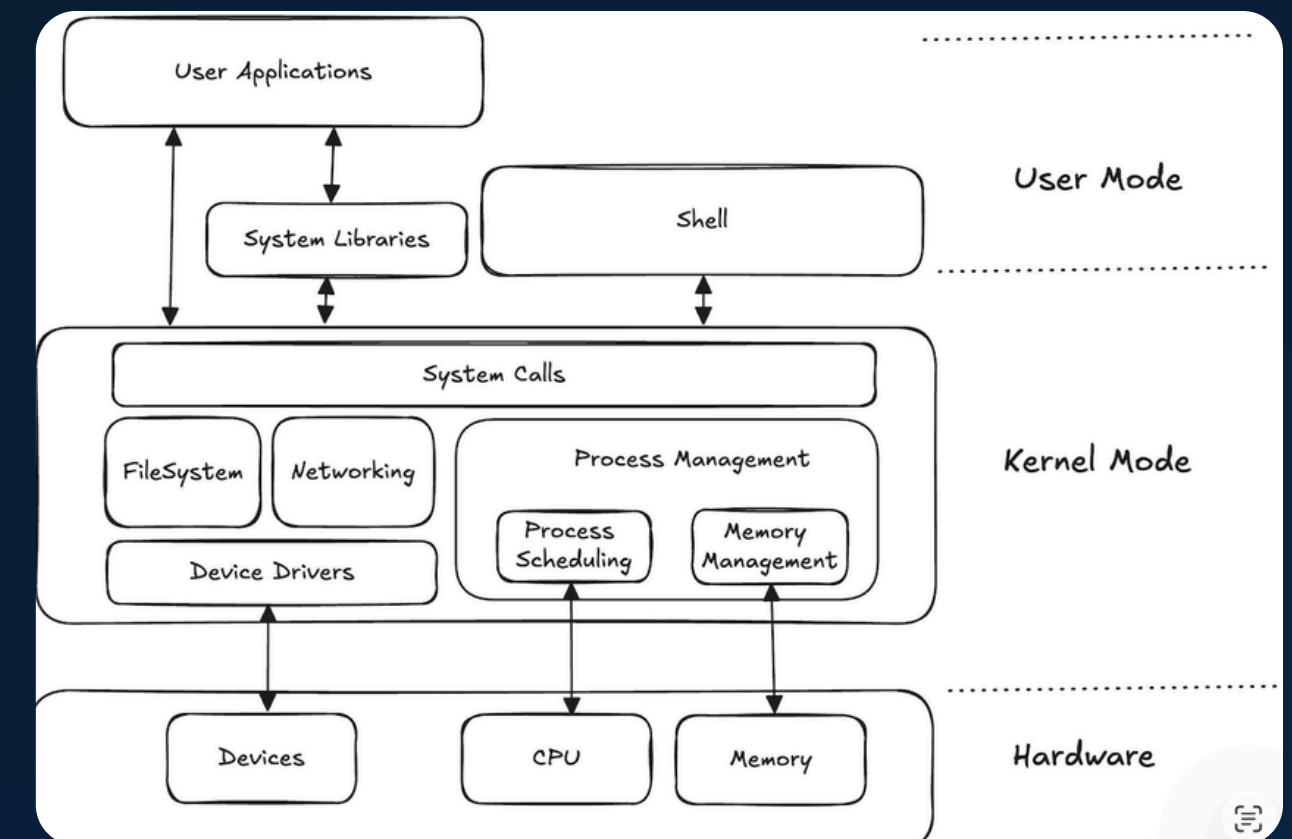
- OS is divided into layers (each layer performs specific functions)
- Each layer interacts only with adjacent layers

Advantages:

- Easy to design and debug
- Better modularity

Disadvantages:

- Slower performance due to layer overhead
- Difficult to define proper layers



Explain the different structures of an Operating System (Monolithic, Layered, Microkernel). Compare their advantages and disadvantages.

3. Microkernel Structure

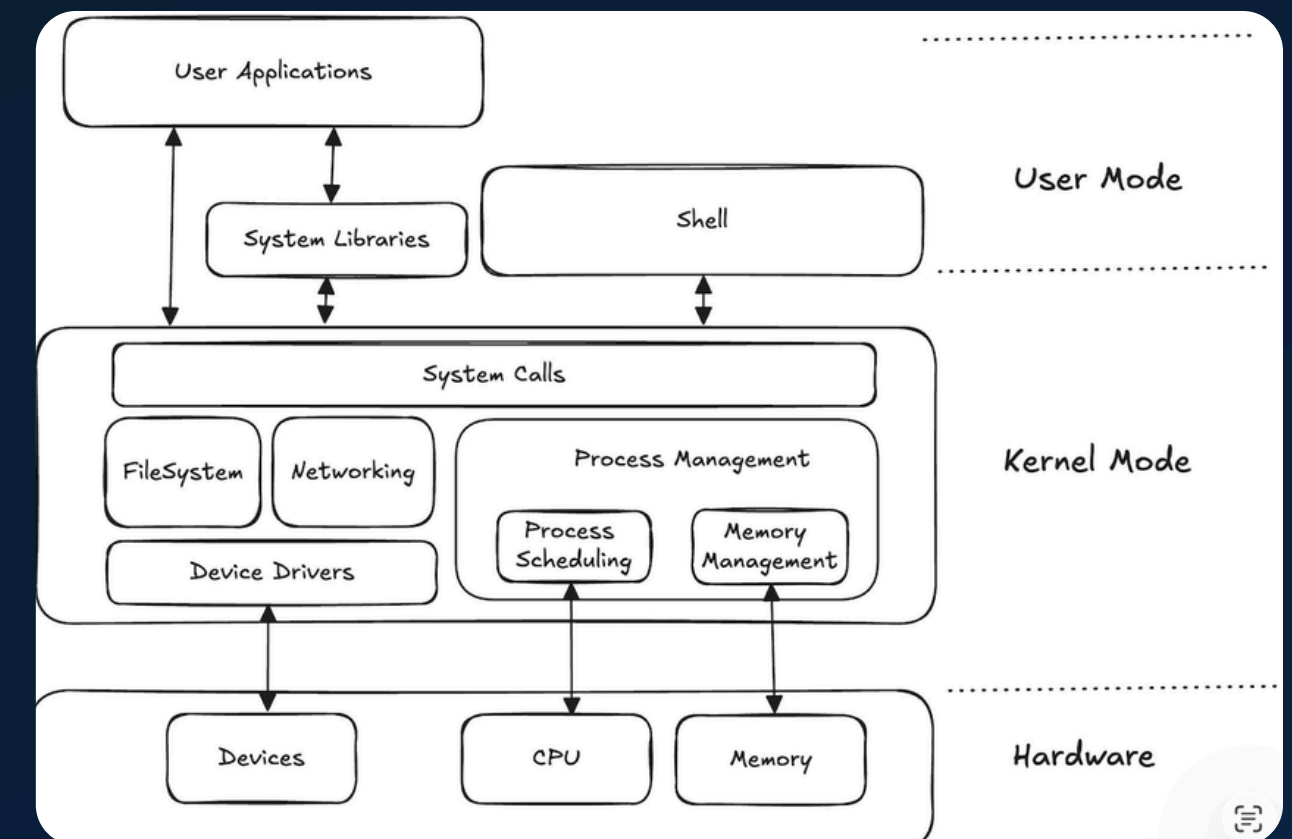
- Only essential services run in kernel (e.g., communication, memory)
- Other services run in user space

Advantages:

- High security and stability
- Easy to extend and maintain

Disadvantages:

- Lower performance (due to message passing)
- More complex communication



What are OS services? Explain different categories of system calls with examples.

OS services are functions provided by the operating system to assist user programs — such as I/O handling, resource allocation, and file management.

Process Control: Create/terminate processes — `fork()`, `exit()`

File Management: Open/close files — `open()`, `close()`

Device Management: Request/release devices — `ioctl()`, `read()`

Information Maintenance: Get/set system data — `getpid()`, `time()`

Communication: Create/delete communication connections — `pipe()`, `shmget()`

These services ensure user programs can interact with hardware safely without direct access.

Explain the concept of Virtual Machine. Discuss different types of VMs and their applications in modern computing.

- A Virtual Machine (VM) is a software-based emulation of a physical computer that runs an operating system and applications as if on real hardware.
- Process VM: Provides a platform-independent execution environment (e.g., JVM for Java bytecode).
- System VM: Emulates complete hardware to run a full guest OS (e.g., VMware, VirtualBox).
- Applications: Server consolidation, sandboxing for security, testing/development environments, and cloud computing.
- Key benefit: Isolation — each VM operates independently, so failures in one do not affect others.



Unit 2

Processes



Define a process. How is it different from a program?

- A process is a program in execution — it includes the program code (text section), current activity (program counter), stack, data section, and heap.
- A program is a passive entity (a file on disk), while a process is an active entity with a program counter and associated resources.
- One program can create multiple processes (e.g., multiple browser tabs running the same web browser executable).
- A process has a Process Control Block (PCB) that stores its state, registers, and scheduling info — a program does not.
- Processes require CPU time, memory, and I/O resources to execute; programs simply occupy storage space.

What is a Process Control Block (PCB)? What information does it contain?

A PCB is a data structure maintained by the OS for each process — it contains all information needed to manage and control the process.

Information contained in PCB:

- Process ID (PID): Unique identifier of the process
- Process State: Current state (new, ready, running, waiting, terminated).
- Program Counter: Address of the next instruction to be executed for this process.
- CPU Registers: Contents of all process-centric registers saved during an interrupt.
- Memory Management Info: Base/limit registers, page tables, or segment tables.
- I/O Status Info: List of I/O devices allocated, open files, and pending I/O requests.

The PCB helps the OS manage and track processes efficiently.

Differentiate between user-level threads and kernel-level threads.

User-level threads are fast but less flexible, while kernel-level threads are slower but more powerful and reliable.

Feature	User-Level Threads	Kernel-Level Threads
Management	Managed by user libraries	Managed by OS kernel
OS Awareness	OS is not aware	OS is aware
Performance	Faster (no system calls)	Slower (requires system calls)
Context Switching	Fast	Slower
Blocking	One thread blocks all	Only blocked thread is affected
Portability	More portable	Less portable

Define CPU scheduling. What are the different scheduling criteria?

CPU scheduling is the process by which the operating system selects one process from the ready queue to allocate the CPU.

Scheduling Criteria:

- CPU Utilization: Maximize CPU usage
- Throughput: Increase number of processes completed
- Turnaround Time: Minimize total execution time
- Waiting Time: Minimize time spent in ready queue
- Response Time: Minimize time to first response

Explain the following CPU scheduling algorithms with examples: FCFS, SJF, and Round Robin. Compare their performance based on waiting time and turnaround time.

1. FCFS (First Come First Serve)

- Processes are executed in the order they arrive
- Example: $P1 \rightarrow P2 \rightarrow P3$

Pros: Simple, easy to implement

Cons: High waiting time (convoy effect)

2. SJF (Shortest Job First)

- Process with smallest execution time is selected first
- Example: If burst times: $P1=6, P2=2, P3=4 \rightarrow$ Order: $P2 \rightarrow P3 \rightarrow P1$

Pros: Minimum waiting & turnaround time

Cons: Difficult to predict burst time, starvation possible

3. Round Robin (RR)

- Each process gets fixed time slice (quantum)
- Example: Time quantum = 2 $\rightarrow P1 \rightarrow P2 \rightarrow P3 \rightarrow P1 \rightarrow \dots$

Pros: Fair scheduling, good for time-sharing

Cons: Higher waiting time if quantum is small

Discuss the different states of a process with a neat diagram. Explain the role of various schedulers (long-term, short-term, medium-term) in process management.

Main States of a Process:

- New – Process is being created
- Ready – Waiting for CPU allocation
- Running – Currently executing on CPU
- Waiting (Blocked) – Waiting for I/O or event
- Terminated – Execution completed

Discuss the different states of a process with a neat diagram. Explain the role of various schedulers (long-term, short-term, medium-term) in process management.

Schedulers in OS

1. Long-Term Scheduler (Job Scheduler)

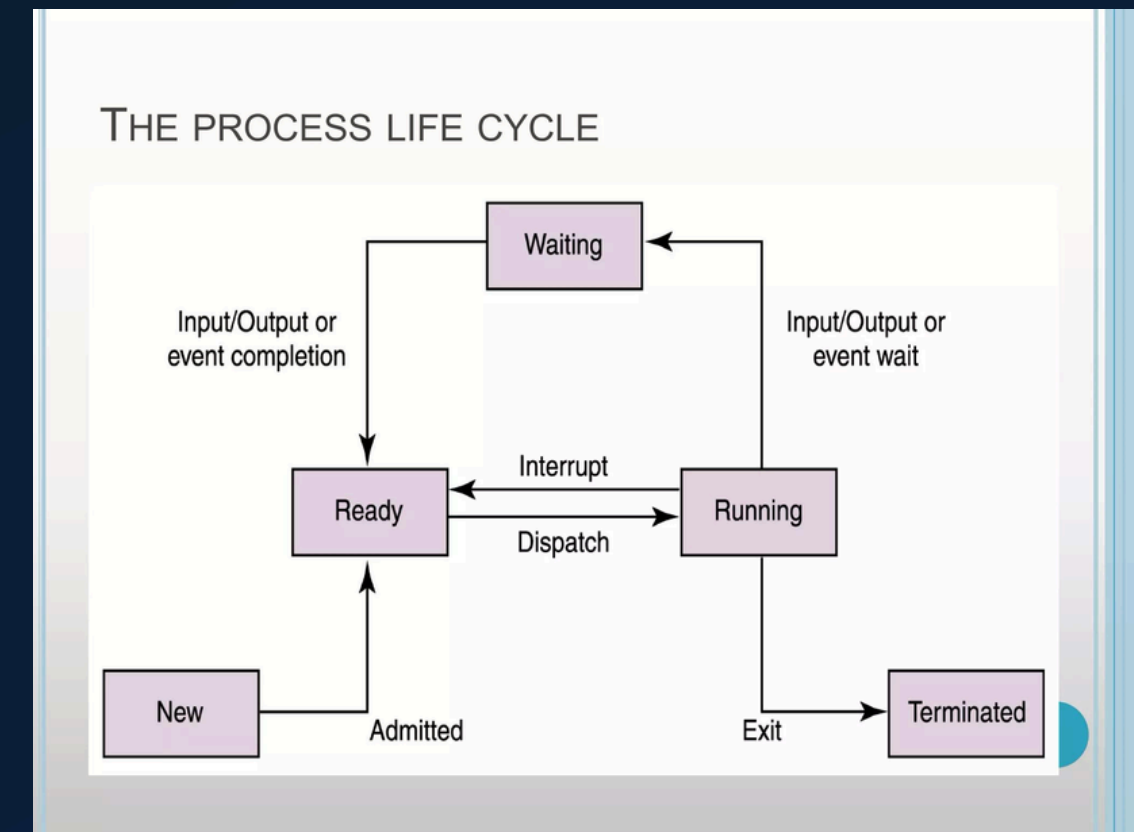
- Selects processes from job pool (new → ready)
- Controls degree of multiprogramming
- Runs less frequently

2. Short-Term Scheduler (CPU Scheduler)

- Selects process from ready queue → CPU (ready → running)
- Runs very frequently
- Must be fast

3. Medium-Term Scheduler

- Temporarily removes processes from memory (swapping)
- Moves processes (ready ↔ waiting/suspended)
- Helps in memory management





Unit 3

Inter-process Communication

What is a critical section problem? State the requirements for a solution.

The critical section problem occurs when multiple processes access shared resources (data/variables) simultaneously, which can lead to inconsistency. The goal is to ensure proper synchronization so only one process executes in the critical section at a time.

Explain Peterson's solution for the critical section problem.

Peterson's solution is a software-based technique to solve the critical section problem for two processes using shared variables. Peterson's solution uses flag + turn to safely control access to the critical section between two processes.

Concept:

Uses two variables:

- flag[2] → indicates if a process wants to enter the critical section
- turn → indicates whose turn it is

Working:

- Both processes set their flag to true
- turn decides who enters first
- One process enters, the other waits

What is a semaphore? Differentiate between binary and counting semaphores.

A semaphore is a synchronization tool used to control access to shared resources in a system using a variable and two operations: wait() and signal().

Feature	Binary Semaphore	Counting Semaphore
Value Range	0 or 1	0 to N
Usage	Single resource	Multiple resources
Purpose	Mutual exclusion	Resource management

Define a monitor. How is it different from a semaphore?

- A monitor is a high-level synchronization construct that encapsulates shared data and the procedures that operate on it within a single module.
- Only one process can be active inside a monitor at a time — mutual exclusion is automatically enforced by the monitor.
- Monitors use condition variables (wait/signal) for synchronization, whereas semaphores use integer variables with P/V operations.
- Semaphores are error-prone (incorrect placement of wait/signal causes bugs); monitors are safer as the compiler enforces access rules.
- Example: Java's synchronized blocks and methods implement monitor-like behavior.

What is busy waiting? What are its disadvantages?

Busy waiting (spinlock) is a technique where a process continuously checks a condition in a loop while waiting for access to a critical section.

- It wastes CPU cycles that could be used by other productive processes — the CPU is occupied but doing no useful work.
- In single-CPU systems, busy waiting is especially harmful because other processes cannot run while the CPU is spinning.
- In multi-CPU systems, spinlocks may be acceptable for very short waits because context-switch overhead exceeds the spin time.

Alternative: Use blocking/waiting primitives (semaphores) that put the process to sleep and wake it when the resource is available.

Explain the Readers-Writers problem.

It is a synchronization problem where multiple processes share a common data resource.

- Readers → Only read data (can access simultaneously)
- Writers → Modify data (require exclusive access)

Allow multiple readers together, but only one writer at a time, with proper synchronization.



Unit 4

Deadlocks



Define deadlock. What are the four necessary conditions for deadlock?

Deadlock is a situation where a set of processes are blocked because each process is waiting for a resource held by another process.

Four Necessary Conditions for Deadlock:

1. Mutual Exclusion

- At least one resource is used by only one process at a time

2. Hold and Wait

- A process holds resources while waiting for others

3. No Preemption

- Resources cannot be forcibly taken; they must be released voluntarily

4. Circular Wait

- A circular chain of processes exists, each waiting for a resource held by the next

What is a safe state in the context of Banker's algorithm?

A safe state is a state of the system where there exists a safe sequence of processes such that all processes can complete their execution without leading to deadlock.

Key Idea:

- The system can allocate resources in some order (safe sequence)
- Each process gets required resources and finishes
- Resources are released and used by next process

How can deadlock be recovered once detected?

Once a deadlock is detected, the system must take actions to break it and resume execution.

Recovery Methods:

1. Process Termination

- Abort all deadlocked processes or
- Abort one process at a time until deadlock is removed

2. Resource Preemption

- Temporarily take resources from some processes
- Allocate them to others to break deadlock

What is the difference between a safe state and an unsafe state?

Feature	Safe State	Unsafe State
Definition	System can allocate resources safely	No guarantee of safe allocation
Deadlock	No deadlock possible	May lead to deadlock
Safe Sequence	Exists	Does not exist
System Condition	Stable and predictable	Risky and uncertain

Explain the four necessary conditions for deadlock. For each condition, describe how it can be prevented. Is it practical to prevent all four conditions simultaneously?

- Mutual Exclusion: Use shareable resources (e.g., read-only files). However, some resources like printers cannot be shared — not always preventable.
- Hold and Wait: Require a process to request all needed resources at once before execution, or release all held resources before requesting new ones.
- No Preemption: If a process cannot get a resource, preempt (forcefully release) its currently held resources and restart it later.
- Circular Wait: Impose a total ordering on resource types and require each process to request resources in increasing order only.
- It is NOT practical to prevent all four simultaneously — preventing mutual exclusion is impossible for non-shareable resources, and hold-and-wait prevention leads to low resource utilization.

What is deadlock detection? Explain the deadlock detection algorithm for multiple instances of each resource type. How does recovery differ from prevention and avoidance?

- Deadlock detection is the process of checking whether the system is currently in a deadlocked state, using algorithms that examine resource allocation graphs or matrices.
- Algorithm: Use data structures similar to Banker's — Available, Allocation, Request. Mark processes with zero allocation; then find a process whose $\text{Request} \leq \text{Work}$, add its allocation to Work, and repeat.
- If any process remains unmarked after the algorithm finishes, those processes are deadlocked.
- Recovery takes action AFTER deadlock occurs (process termination or resource preemption), while prevention ensures deadlock cannot occur and avoidance ensures the system stays in a safe state.
- Detection + recovery is used when deadlocks are rare; the overhead of detection is incurred only when needed, unlike prevention/avoidance which impose overhead on every request.



Unit 5

Memory Management



Differentiate between logical and physical addresses.

- A logical address is generated by the CPU during program execution; it is the address seen by the process (also called virtual address).
- A physical address is the actual address in main memory (RAM) — it is the address seen by the memory hardware.
- The Memory Management Unit (MMU) translates logical addresses to physical addresses at runtime using a relocation register or page table.
- Logical addresses are in the range 0 to MAX (process address space); physical addresses are 0 to MAX (physical memory size).
- The user/programmer only deals with logical addresses — the OS handles the mapping transparently.

Define paging. What problem does it solve?

Paging is a memory management technique in which logical memory is divided into fixed-size pages and physical memory into frames, and pages are mapped to frames.'

Problem it Solves:

- Eliminates external fragmentation
- Allows non-contiguous memory allocation
- Improves efficient use of memory

What is a page fault? How is it handled?

A page fault occurs when a process tries to access a page that is not present in main memory (RAM).

How it is Handled:

1. Trap to OS – Control goes to operating system
2. Check Validity – If reference is valid → continue, else → error (invalid access)
3. Find Page on Disk – Locate required page in secondary memory
4. Load Page into RAM – Bring page into a free frame (or replace another page)
5. Update Page Table – Mark page as present
6. Restart Instruction – Resume execution

What is thrashing? How can it be prevented?

- Thrashing occurs when a process spends more time paging (swapping pages in and out) than executing — CPU utilization drops dramatically.
- It happens when the degree of multiprogramming is too high and processes do not have enough frames to hold their working sets.
- The working set model prevents thrashing by monitoring each process's set of recently used pages and ensuring enough frames are allocated.
- The OS can also use the page fault frequency (PFF) strategy — if a process faults too frequently, allocate more frames; if faulting rarely, deallocate some.
- Reducing the number of active processes (lowering multiprogramming) is the most direct way to stop thrashing.

Differentiate between paging and segmentation.

- Paging divides memory into fixed-size blocks (pages/frames); segmentation divides memory into variable-sized logical units (segments) like code, stack, data.
- Paging is hardware-driven — the page size is fixed by the system; segmentation is software-driven — segment sizes depend on the logical structure of the program.
- Paging eliminates external fragmentation but may cause internal fragmentation; segmentation eliminates internal fragmentation but may cause external fragmentation.
- In paging, the address is (page number, offset); in segmentation, the address is (segment number, offset) with bounds checking.
- Modern systems often use combined paging + segmentation — segments are divided into pages to get benefits of both.

Explain the concept of locality of reference.

- Locality of reference is the tendency of a program to access a relatively small set of memory locations repeatedly within a short time period.
- Temporal Locality: If a memory location is accessed, it is likely to be accessed again soon (e.g., loops, counters, stack).
- Spatial Locality: If a memory location is accessed, nearby locations are likely to be accessed soon (e.g., array traversal, sequential code execution).
- Locality is the reason why caching and virtual memory work — recently/frequently used data is kept in faster storage.
- Working set: The set of pages a process needs within a time window — determined by the locality of the process.

Explain the paging scheme in detail. Describe the hardware support for paging, page table structure, and how logical address is translated to physical address.

- Paging divides logical memory into pages (fixed size, e.g., 4KB) and physical memory into frames of the same size. Each page maps to one frame.
- Hardware support: MMU contains a Page Table Base Register (PTBR pointing to page table) and Page Table Length Register (PTLR for bounds checking).
- Translation: Logical address = (page number p , offset d). Use p to index into the page table to get frame number f . Physical address = (f , d).
- TLB (Translation Lookaside Buffer): A hardware cache for recent page-to-frame mappings — speeds up translation by avoiding page table lookups.
- For multi-level paging: The page number is split into multiple levels; each level indexes into a page table — this reduces the memory needed for large page tables.

What is virtual memory?

Virtual memory is a memory management technique that allows a program to execute even if it is not completely loaded into main memory (RAM) by using secondary storage (disk).

Key Points:

- Uses disk as an extension of RAM
- Only required pages are loaded into memory
- Gives illusion of large memory

Discuss the concept of thrashing. Explain the working set model and how it helps in preventing thrashing. What is the relationship between degree of multiprogramming and CPU utilization?

- Thrashing: When a process spends more time paging than executing, CPU utilization collapses — the system is overwhelmed with page faults.
- Working Set Model: The working set $W(t, \delta)$ is the set of pages a process has referenced in the recent time window δ . Each process needs enough frames for its working set.
- Prevention: If total working sets of all processes exceed available frames, the OS should suspend (swap out) a process rather than let all processes thrash.
- CPU Utilization vs. Multiprogramming: Initially, increasing multiprogramming increases CPU utilization; beyond a threshold, thrashing causes utilization to drop sharply.
- The key is to maintain multiprogramming at the level where the sum of working sets fits in available memory — this is the optimal operating point.



Unit 6

File Management



What is a file? Explain different types of files.

A file is a collection of related data stored on secondary storage, identified by a name.

Types of Files:

1. Regular Files

- Store user data (text, images, programs)

2. Directory Files

- Contain information about files and directories

3. Special Files

- Used for device communication
- Character Special Files – Handle data as stream (e.g., keyboard)
- Block Special Files – Handle data in blocks (e.g., disk)

Explain contiguous, linked, and indexed file allocation methods.

1. Contiguous Allocation

- Files are stored in continuous memory blocks

Advantages:

- Fast access (sequential & direct)
- Simple implementation

Disadvantages:

- External fragmentation
- Difficult to expand file

2. Linked Allocation

- File is stored as linked list of blocks

Advantages:

- No external fragmentation
- Easy file expansion

Disadvantages:

- Slow access (no direct access)
- Pointer overhead

Explain contiguous, linked, and indexed file allocation methods.

3. Indexed Allocation

- Uses an index block to store addresses of all file blocks

Advantages:

- Supports direct access
- No external fragmentation

Disadvantages:

- Extra memory for index block
- Slight overhead

What is a directory? Explain different directory structures.

- A directory is a container that maps file names to their metadata (location, size, permissions, timestamps) and organizes files on a storage device.
- Single-Level: All files in one directory — simple but causes naming conflicts when many files exist.
- Two-Level: Separate directory for each user — solves naming conflicts but makes sharing and grouping difficult.
- Tree-Structured: Hierarchical directories with subdirectories — most common; allows efficient organization and path-based access.
- Acyclic-Graph: Allows shared files/subdirectories via links — supports sharing but complicates deletion (reference counting needed).

Compare FCFS and SSTF disk scheduling algorithms.

Feature	FCFS (First Come First Serve)	SSTF (Shortest Seek Time First)
Working	Serves requests in arrival order	Selects request with minimum seek distance
Performance	Higher seek time	Lower seek time (better performance)
Starvation	No starvation	Possible starvation
Fairness	Fair (first come, first served)	Not always fair
Complexity	Simple	More complex

Explain the SCAN and C-SCAN disk scheduling algorithms.

SCAN (Elevator Algorithm):

The disk arm moves in one direction, servicing all requests along the way. When it reaches the end, it reverses direction and continues servicing requests.

C-SCAN (Circular SCAN):

The disk arm moves in one direction only, servicing requests. After reaching the end, it jumps back to the beginning and continues in the same direction.

Explain the three file allocation methods (contiguous, linked, indexed) in detail.

- Contiguous: Files stored in consecutive blocks. Sequential access is fastest; random access is also fast (direct calculation). Suffers from external fragmentation; file size must be known in advance.
- Linked: Files stored as linked blocks via pointers. No external fragmentation; files can grow dynamically. Random access is very slow ($O(n)$ traversal); pointers consume space in each block.
- Indexed: An index block stores pointers to all data blocks. Fast random access ($O(1)$ via index); no external fragmentation. Large files need multiple index blocks (linked index or multi-level index).

Discuss different directory structures (single-level, two-level, tree-structured, acyclic-graph). Explain their advantages and disadvantages.

- Single-Level: One directory for all files. Simple to implement; but causes naming conflicts and is unmanageable with many files.
- Two-Level: One directory per user. Solves naming conflicts between users; but files cannot be logically grouped within a user's directory.
- Tree-Structured: Hierarchical with subdirectories. Most widely used; allows logical organization via paths. Deleting shared files is problematic — only one path to a file.
- Acyclic-Graph: Allows shared subdirectories/files via links (like Unix symlinks). Enables sharing without duplication; but deletion is complex (reference counting or garbage collection needed).

Describe the principles of I/O software. Explain the layered structure of I/O software including interrupt handlers, device drivers, and device-independent I/O software.

Principles of I/O Software

- Device Independence – Same interface for different devices
- Uniform Naming – Devices accessed like files
- Error Handling – Detect and handle I/O errors
- Buffering – Temporary storage for smooth data transfer
- Efficiency – Optimize performance and CPU usage

Layered Structure of I/O Software

1. User-Level I/O

- Provides interface to users (system calls, libraries)

2. Device-Independent I/O Software

- Common functions for all devices
- Examples: buffering, caching, error handling

3. Device Drivers

- Specific to each device
- Converts OS commands into device-specific operations

4. Interrupt Handlers

- Handles interrupts from devices
- Notifies CPU after I/O completion

THANK YOU !!